KNIT CHATBOT

Informational Module

Technical Documentation

Repository: KNIT-CHATBOT

`Module: src/ai_engine/informational/`

Branch: dev

1. Executive Summary

The Informational module is a multi-domain knowledge retrieval system that routes user queries across five intent categories: HANDBOOK, FAQS, PROCESS, HOLIDAYS, and UNKNOWN. It uses an LLM-based intent classifier to determine the correct knowledge domain, then applies a domain-specific RAG (Retrieval-Augmented Generation) pipeline or a live API call to generate an accurate, context-aware answer.

Key Capabilities

- LLM-powered intent classifier (InformationClassifier) with detailed category definitions, boundary rules, and a priority tie-breaker
- Per-category RAG handlers (FAQSHandler, HandbookHandler, ProcessHandler) using MultiQueryRetriever, FlashRankReranker, and gpt-4.1-mini for answer generation
- Live holiday data retrieval from the HR Hub REST API (HolidayFetcher) for the HOLIDAYS category
- Cross-module routing: UNKNOWN queries return a suggested_module hint so ChatService can reroute to Analytical, Operational, Workflow, or DailyDigest
- Multi-turn conversation context via a shared ConversationHistory instance, with pronoun-resolution rewriting before retrieval
- Lightweight native router (InformationalNativeRouter) for MetaLayer-aware mid-conversation routing
- TTL-based in-memory cache (SHA-256 key per category + resolved query) to avoid redundant retrieval on repeated questions

2. System Overview

The Informational module is structured in three logical layers:

Field	Description
Classification Layer	InformationClassifier assigns each incoming query to exactly one of five categories using an LLM call with structured rules.
Retrieval Layer	Per-category RAG handlers query a shared Pinecone index (knit-information) across dedicated namespaces. HolidayFetcher handles HOLIDAYS via REST API.
Generation Layer	Each handler calls gpt-4.1-mini with retrieved context to produce the final natural-language response.

Key Source Files

```
src/ai_engine/informational/processor.py          — InformationalProcessor (main orchestrator)
src/ai_engine/informational/classifier.py         — InformationClassifier (LLM router)
```

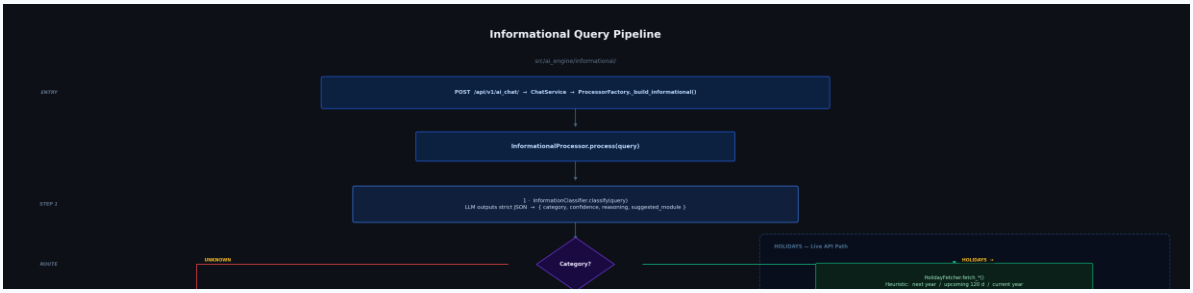
src/ai_engine/informational/classifier_rules.py — Category definitions, boundary rules, priority note

```
src/ai_engine/informational/handler/base_handler.py — BaseRAGHandler (abstract)
src/ai_engine/informational/handler/faqs_handler.py — FAQSHandler
src/ai_engine/informational/handler/handbook_handler.py — HandbookHandler
src/ai_engine/informational/handler/process_handler.py — ProcessHandler
src/ai_engine/informational/holidays/holiday_fetcher.py — HolidayFetcher
src/ai_engine/informational/native_router.py      — InformationalNativeRouter
knowledge_base/data/faq/                          — Source PDF documents for all RAG categories
```

3. Data Flow

3.1 Classification Pipeline

Every query follows this top-level routing path:



1. User submits a natural-language query via POST /api/v1/ai_chat/
2. ChatService routes to InformationalProcessor.process(query, conversation_id)
3. InformationClassifier.classify(query) calls gpt-4.1-mini with category definitions, boundary rules, and priority note
4. Returns: { category, confidence (0.0–1.0), reasoning, suggested_module }
5. UNKNOWN guard: if category == UNKNOWN, return out-of-scope message with suggested_module hint
6. HOLIDAYS: HolidayFetcher fetches live data from HR Hub API, formats as context, LLM generates answer
7. FAQS / HANDBOOK / PROCESS: handler-based RAG pipeline (see 3.2)

3.2 RAG Pipeline (FAQS / HANDBOOK / PROCESS)

Each domain handler runs the same retrieve-then-generate pattern:

8. Pronoun resolution: LLM rewrites the query using recent conversation history (e.g. 'his department' → 'John Doe's department')
9. Cache check: SHA-256(category + resolved_query) looked up in TTL-based in-memory cache
10. MultiQueryRetriever: generates N variant phrasings of the query, fetches a candidate pool from Pinecone
11. FlashRankReranker: local cross-encoder scores all candidates and re-ranks them in a single inference pass
12. Threshold filter: only documents above similarity_threshold pass; fallback pads from ranked pool if too few qualify
13. LLM answer generation: gpt-4.1-mini receives the filtered context and produces a grounded response
14. Turn persisted to ConversationHistory (user query + assistant response)

3.3 Holiday Lookup Pipeline

15. HolidayFetcher.fetch_year() or fetch_upcoming() calls the HR Hub REST API with the Bearer token
16. Date range heuristics: 'next year' → annual, 'upcoming' → 120-day window, else → current year
17. format_for_context() converts API response to structured text for the LLM prompt
18. gpt-4.1-mini generates the final answer from the formatted holiday data

4. Component Analysis

4.1 InformationClassifier

Location: src/ai_engine/informational/classifier.py

LLM-based router that assigns each query to one of five categories. All classification rules are defined in classifier_rules.py and injected into the prompt at runtime.

Field	Description
Model	gpt-4.1-mini (configurable via classifier_model param)
Output schema	{ category: str, confidence: float, reasoning: str, suggested_module: str }
Parse failure	Defaults to FAQS with confidence=0.40
Low confidence	Warning included if confidence < 0.60

4.2 BaseRAGHandler (Abstract)

Location: src/ai_engine/informational/handler/base_handler.py

Abstract parent class for FAQSHandler, HandbookHandler, and ProcessHandler. Defines the full retrieval pipeline and exposes retrieve() and get_context_for_llm().

Configuration Fields (all subclass-overridable):

Field	Description
`chunk_size / chunk_overlap`	Controls PDF text splitting for vectorstore ingestion
`embedding_dimensions`	512 (text-embedding-3-small)
`top_k`	Number of candidates retrieved from Pinecone before reranking
`similarity_threshold`	Minimum relevance score for documents to pass the filter
`cache_ttl_seconds`	TTL for the in-memory query result cache

4.3 FAQSHandler

Location: src/ai_engine/informational/handler/faqs_handler.py

- Corpus: Platform features, integrations, technical Q&A, KNIT system functionality
- Pinecone namespace: PINECONE_INFORMATIONAL_NS_FAQS (default: faqs)
- Config: chunk_size=1000, overlap=200, top_k=5, similarity_threshold=0.30

4.4 HandbookHandler

Location: src/ai_engine/informational/handler/handbook_handler.py

- Corpus: Employee benefits, culture, org structure, HR contacts, policies, entitlements
- Pinecone namespace: PINECONE_INFORMATIONAL_NS_HANDBOOK (default: handbook)
- Config: chunk_size=1500, overlap=300, top_k=5, similarity_threshold=0.30
- Additional processing: extracts contact info (emails, phones) from retrieved documents; confidence scoring (high >= 0.8, medium >= 0.65, low < 0.65)

4.5 ProcessHandler

Location: src/ai_engine/informational/handler/process_handler.py

- Corpus: Step-by-step how-to guides, system navigation paths, KNIT workflow instructions
- Pinecone namespace: PINECONE_INFORMATIONAL_NS_PROCESS (default: process)
- Config: chunk_size=800, overlap=150, top_k=5, similarity_threshold=0.30
- Additional processing: extracts numbered steps from LLM response; injects visual-aid tip when response contains more than 3 steps; confidence scoring (high >= 0.85, medium >= 0.70, low < 0.70)

4.6 HolidayFetcher

Location: src/ai_engine/informational/holidays/holiday_fetcher.py

- fetch_year(year) — Fetches all holidays for a given calendar year
- fetch_upcoming(days=120) — Fetches holidays within a rolling window
- format_for_context() — Converts API response into structured text for the LLM prompt

- Authentication: Bearer token passed from the authenticated user session
- Date range heuristics: 'next year' → annual fetch; 'upcoming' → 120-day window; else → current year

4.7 InformationalProcessor

Location: src/ai_engine/informational/processor.py

Main orchestrator. Receives query from ChatService, runs the full classification-to-response pipeline.

Field	Description
token: str	Bearer token for HR Hub API calls (HolidayFetcher)
`user_id: int | None`	Used for conversation history keying
`custom_configs: dict[str, RetrievalConfig] | None`	Override per-category RAG settings
`classifier_model: str`	LLM model for classification (default: gpt-4.1-mini)

Handler initialization: Lazy (first use), async-locked to prevent race conditions. Initialized handlers are cached in _handlers dict.

4.8 InformationalNativeRouter

Location: src/ai_engine/informational/native_router.py

Lightweight adapter for MetaLayer-aware mid-conversation routing. Called by ChatService when an Informational session is active and a new message arrives, avoiding a full LLM reclassification on every turn.

- Calls InformationClassifier.classify() on the incoming query
- Returns RouterDecision(decision='related', confidence=0.9) if category != UNKNOWN
- Returns RouterDecision(decision='unrelated', confidence=0.8, suggested_processor=...) if category == UNKNOWN
- Fails open on httpx/parse errors (confidence=0.5) to avoid false reroutes

5. How It Works: Step-by-Step

5.1 Query Classification

19. ChatService.process_query() determines intent = Informational and calls ProcessorFactory.get_processor('Informational', user=user)
20. InformationalProcessor.process(query, conversation_id) is called
21. InformationClassifier.classify(query) sends query + full rules to gpt-4.1-mini
22. Returns { category, confidence, reasoning, suggested_module }
23. If UNKNOWN: log warning, return fallback message with suggested_module in classification_warning field

5.2 RAG Retrieval and Answer Generation

24. Get or initialize the domain handler (FAQSHandler / HandbookHandler / ProcessHandler)
25. Resolve pronouns: LLM rewrites query using last N conversation turns
26. Check in-memory cache: if hit, return cached context immediately
27. MultiQueryRetriever generates N query phrasings, fetches candidates from Pinecone
28. FlashRankReranker scores and reranks all candidates in one pass
29. Threshold filter keeps only docs above similarity_threshold; fallback pads if too few pass
30. gpt-4.1-mini generates answer grounded in filtered context
31. ConversationHistory.add_entry() persists the turn
32. Build and return full response dict with classification metadata, latency breakdown, and document list

5.3 Holiday Lookup

33. HolidayFetcher determines date range from query heuristics
34. Calls HR Hub REST API with Bearer token
35. format_for_context() formats the API response
36. gpt-4.1-mini generates the final answer from the formatted data
37. Response returned with category=HOLIDAYS and classification metadata

6. Classification Rules

6.1 Category Definitions

HANDBOOK — Employee Handbook

Scope: All information in the Employee Handbook — benefits overview, company culture, organizational structure, office amenities, HR contacts, official rules and regulations, entitlements, compliance requirements, legal guidelines, disciplinary procedures, data-privacy rules, and formal policy statements.

Key signal: The user wants to KNOW a rule, entitlement, or policy — not how to perform an action in a system.

- Examples: 'When do health benefits start?', 'What is the leave policy?', 'What is the code of conduct?', 'How many sick leave days am I entitled to?'
- Anti-examples (redirect to PROCESS): 'How do I file a leave request in KNIT?', 'How do I update my profile in KNIT?'

FAQS — Frequently Asked Questions

Scope: Common questions answerable from static knowledge — platform documentation, general HR definitions, or system feature descriptions. Must NOT require a live database query or real-time employee records.

Key signal: Short, factual answer that can be derived from a static document or system description.

- Examples: 'What is KNIT?', 'What integrations does KNIT support?', 'Can I access KNIT on mobile?', 'What is the 13th month pay?'
- Anti-examples (redirect to UNKNOWN/Analytical): 'How many employees are on leave today?', 'Who was absent yesterday?'

PROCESS — How-to Guides and KNIT Workflows

Scope: Step-by-step instructions for completing tasks inside the KNIT system or following an HR workflow. Identified by action intent — the user wants to DO something, not just KNOW something.

Key trigger phrases: 'how do i', 'how to', 'steps to', 'where do i', 'where can i', 'guide me', 'walk me through', 'show me how'

- Examples: 'How do I file a leave request in KNIT?', 'How do I download my payslip?', 'Steps to update my emergency contact'

HOLIDAYS — Live Holiday Lookup

Scope: Questions requiring a lookup of specific holiday dates from the HR system. The user wants to know WHEN a holiday falls, WHICH holidays occur in a period, or WHETHER a specific date is a holiday.

Distinct from HANDBOOK: HOLIDAYS is about concrete dates and schedules, not about holiday policy or entitlements.

- Examples: 'When is the next non-working holiday?', 'List all holidays this year', 'Is November 1 a holiday?'
- Anti-examples (redirect to HANDBOOK): 'How many holiday days are we entitled to?', 'What is the company holiday policy?'

UNKNOWN — Out of Scope

Used when the query requires live HR data, employee records, or actions that cannot be answered from static documents or how-to guides.

Key signal: Answering the question requires querying a live database or performing an HR transaction.

Field	Description
Analytical	Live employee/HR data (headcount, attendance, leave balances, etc.)
Workflow	Ticket or task creation
Operational	HR actions (leave filing, schedule changes, time adjustments)
`DailyDigest`	Summaries and digests
Empty	Completely out of scope

6.2 Boundary Rules

Field	Description
"How do I request leave?"	→ PROCESS (user wants to DO it in the system)
"How many leave days do I have?"	→ HANDBOOK (entitlement defined in handbook)
"What is the leave policy?"	→ HANDBOOK (asking for the rule document)
"When is the next non-working holiday?"	→ HOLIDAYS (live date lookup from HR system)
"How many holiday days are we entitled to?"	→ HANDBOOK (entitlement rule in policy)
"How many employees are on leave this week?"	→ UNKNOWN (live HR data → Analytical)
"Who was absent yesterday?"	→ UNKNOWN (attendance data → Analytical)
"File an overtime request for me"	→ UNKNOWN (HR action → Operational)
"Give me my daily summary"	→ UNKNOWN (daily digest → DailyDigest)

6.3 Priority Order

When a query fits more than one category:

- Standard priority: PROCESS > HANDBOOK > FAQS
- CRITICAL OVERRIDE: Choose UNKNOWN regardless of the above order when the query requires a live database lookup or real-time employee data.
- EXCEPTION: Holiday date lookups are always HOLIDAYS — even though they require live data.

7. Response Schema

InformationalProcessor.process() returns a dict with the following fields:

Field	Description
`conversation_id: str`	Unique identifier for this query turn
query: str	Original user query
response: str	Generated answer
`graphql_query: str`	Always 'N/A' for Informational
`graphql_result: str`	Always 'N/A' for Informational
`rendered_table: None`	Always None for Informational
`rendered_diagrams: None`	Always None for Informational
`from_history: bool`	Whether answer was served from cache
category: str	Classifier output (HANDBOOK / FAQS / PROCESS / HOLIDAYS / UNKNOWN)

`classification_confidence: float`	Classifier confidence score (0.0–1.0)
`classification_reasoning: str`	Classifier's reasoning string
`classification_warning: str | None`	Warning if confidence < 0.60 or category=UNKNOWN
confidence: str	Human-readable tier: 'high', 'medium', or 'low'
`total_found: int`	Number of documents retrieved
cached: bool	Whether result was served from in-memory cache
`latency_ms: dict`	Per-stage timing: { retrieve_ms, rerank_ms, total_ms }
`documents: list[dict]`	Retrieved docs: [{ source, page, relevance_score, excerpt }]

8. Configuration & Environment Variables

All settings are defined in src/conf.py (Pydantic BaseSettings) and loaded from .env.

Pinecone / Retrieval

Field	Description
PINECONE_API_KEY_INFORMATIONAL	API key for the informational Pinecone project
PINECONE_INDEX_INFORMATIONAL	Index name (default: knit-information)
PINECONE_INFORMATIONAL_NS_FAQS	Namespace for FAQS category (default: faqs)
PINECONE_INFORMATIONAL_NS_HANDBOOK	Namespace for HANDBOOK category (default: handbook)
PINECONE_INFORMATIONAL_NS_PROCESS	Namespace for PROCESS category (default: process)

LLM / Embeddings

Field	Description
OPENAI_API_KEY	OpenAI API key
OPENAI_MODEL	Default LLM model (default: gpt-4.1-mini)
OPENAI_EMBEDDING_MODEL	Embedding model (default: text-embedding-3-small)
OPENAI_EMBEDDING_DIMENSIONS	Embedding dimensions (default: 512)

Routing Confidence Thresholds

Field	Description
NATIVE_ROUTER_CONFIDENCE_RELATED	Confidence when query is related (default: 0.9)

NATIVE_ROUTER_CONFIDENCE_UNRELATED	Confidence when query is unrelated (default: 0.8)
NATIVE_ROUTER_CONFIDENCE_ERROR_FALLBACK	Confidence on router error/fail-open (default: 0.5)

HR Hub API

Field	Description
HR_HUB_API_BASE_URL	Base URL for live holiday and HR data (default: https://hrhub.idealforliving.com)

9. Integration with ChatService

9.1 ProcessorFactory Registration

InformationalProcessor is registered in src/ai_engine/factory.py as the handler for intent_type='Informational'. ProcessorFactory._build_informational(user) constructs it with token=user.token and user_id=user.id.

9.2 MetaLayer and Cross-Module Routing

When an Informational session is active (MetaLayer.current_intent='Informational'), ChatService calls InformationalNativeRouter.route(query) on each new message.

- If route() returns 'related': query is handled by InformationalProcessor (no LLM reclassification)
- If route() returns 'unrelated': ChatService reroutes to the suggested_processor using _handle_standard_reroute()

InformationalProcessor itself returns UNKNOWN with a suggested_module in the response dict. ChatService reads this and updates MetaLayer.current_intent accordingly for the next turn.

9.3 Conversation History

InformationalProcessor receives an optional ConversationHistory instance shared across all modules for the same user/thread. It reads recent history for pronoun resolution and writes each completed turn with intent_type=category.

- History keyed by user token + thread_id
- Max 20 entries stored (deque maxlen)
- Persisted to disk as JSON at conversations_dir/{user_token}_conversation_history.json
- Recent 3 entries used by default for pronoun-resolution context

10. Knowledge Base Management

Source PDFs live in knowledge_base/data/faq/ and are organized by category:

Field	Description

FAQS / General	Knit.pdf, KNIT faqs.pdf, Sourcefit IT Support FAQs.pdf
HANDBOOK	Sourcefit_Policy_Manual.pdf
PROCESS	Knit - How To Guides/ (subdirectories by feature area: HR Hub, Dashboard, Chat, Projects, Tickets, Activity Feed)

To add new documents:

38. Place the PDF in the correct subdirectory under knowledge_base/data/faq/
39. Run the vectorstore ingestion script to chunk, embed, and upsert to Pinecone
40. PDF best practices: use text-based PDFs (not scanned images), clear heading structure, avoid excessive tables, no duplicate content
41. Optimal chunk parameters: chunk_size 800–1500, overlap 150–300 (varies per handler)

11. Error Handling and Fallbacks

Field	Description
Classification parse failure	Defaults to FAQS with confidence=0.40
Low classification confidence (< 0.60)	Warning included in classification_warning field
No documents above threshold	Falls back to LLM general knowledge; warning logged
`Below minimum docs after filter`	Pads from ranked pool to meet minimum
Pronoun resolution failure	Returns the original unmodified query
`HolidayFetcher API error`	Exception propagated with error detail
Handler initialization failure	RuntimeError raised (caught by ProcessorFactory)
Async lock contention	Concurrent init requests for the same handler wait; only one initializes

12. Troubleshooting Guide

Query Classified to Wrong Category

- Check classification_confidence and classification_reasoning in the response
- If confidence is low (< 0.60), the boundary between categories may be ambiguous — review classifier_rules.py BOUNDARY_RULES
- Update CATEGORY_DEFINITIONS or BOUNDARY_RULES in classifier_rules.py to clarify the distinction

No Documents Returned for a Valid Query

- Check that PDFs are correctly ingested (vectorstore script ran without errors)
- Verify the correct Pinecone namespace is configured for the category

- Lower similarity_threshold in the handler config if relevant docs are being filtered out

Holiday Data Not Returned

- Verify HR_HUB_API_BASE_URL is set and reachable
- Check that the user's Bearer token is valid (token is passed from auth)
- Check HolidayFetcher logs for HTTP error codes

Cross-Module Rerouting Not Working

- Verify InformationalNativeRouter is registered in ChatService._build_native_router()
- Check that the suggested_processor returned by InformationalNativeRouter matches a registered ProcessorFactory intent
- Confirm MetaLayer is being passed correctly in the request